Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos (DOPO) – Grupo 501

Laboratorio 6

Persistencia

Integrantes

Juan David Espitia

Cristian Camilo Salamanca

Profesora

María Irma Díaz Rozo

Bogotá D.C., 2026-2

Índice

1. Parte I. Creando la maqueta **2**

1.1. A. Safari inicial 2

1.2. B. Maqueta: MVC 3

2. Parte II. Implementando persistencia **9**

2.1. A. Salvar y abrir 9

2.2. B. Exportar e importar 11

2.3. C. Comportamiento 14

3. Parte III. Fortaleciendo la persistencia **16**

3.1. A. Guardar y abrir 16

3.2. B. Exportar e importar 17

3.3. Pruebas de unidad 18

4. Parte IV. Bono: perfeccionando importar **18**

4.1. A. Minicompilador 18

4.2. B. Minicompilador flexible 21

5. Retrospectiva **22**

Parte I. Creando la maqueta

A. Safari inicial

1. *¿Cuál ambiente seleccionaron? ¿Por qué?*

   R/ Inicialmente se usó BlueJ por un inconveniente con la importación de fuentes. Sin embargo, se eligió VS Code como ambiente principal, ya que permite un desarrollo orientado a objetos más profesional: navegación entre clases, refactorización y ejecución integrada de pruebas JUnit.

2. *Ejecuten el programa y capturen la pantalla.*

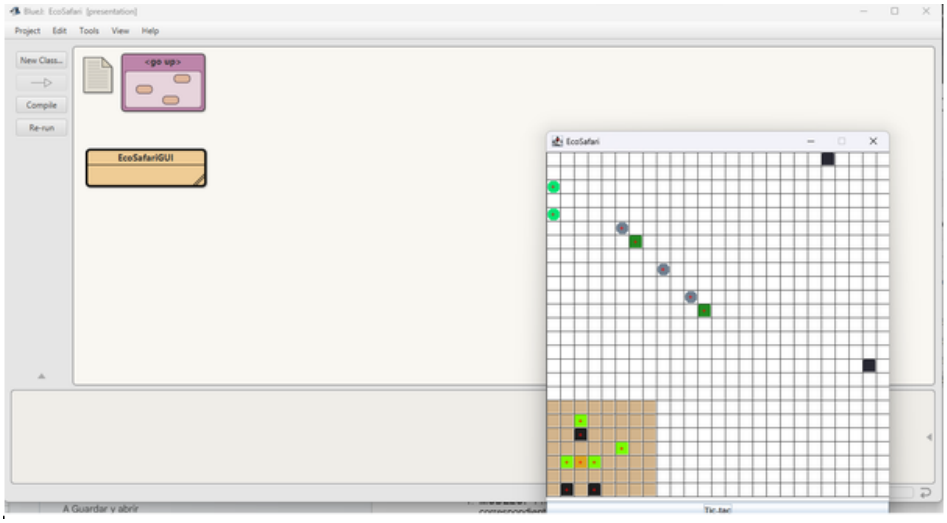


Figura 1: Ejecución inicial de EcoSafari.

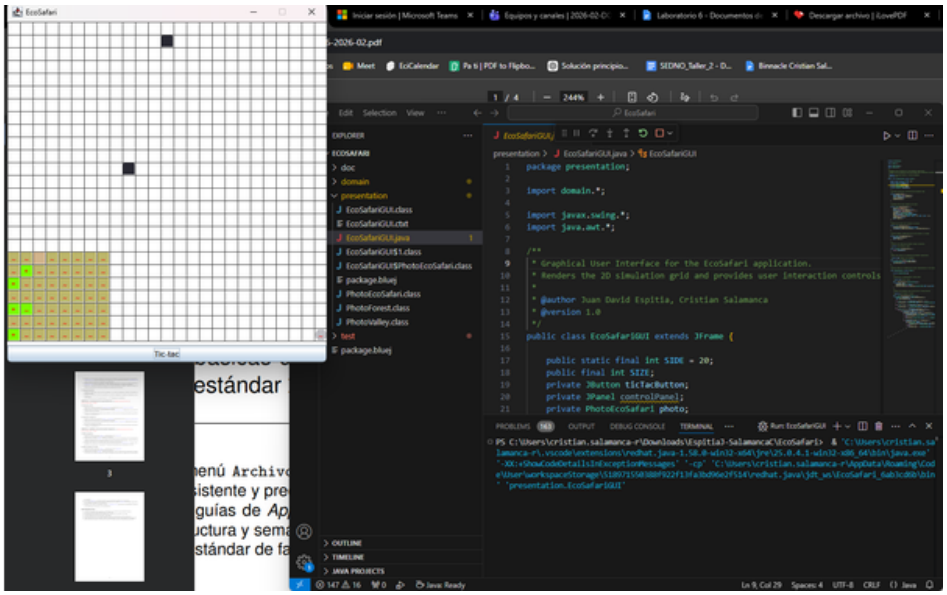


Figura 2: Ejecución de EcoSafari después de algunos clics.

B. Maqueta: MVC

1. **Modelo.** En la fachada `EcoSafari` se crearon los métodos `open`, `saveAs`, `import_` y `exportAs`, que reciben un `File` y propagan una `EcoSafariException`:

```
public void open(File file) throws EcoSafariException {
    String fileName = (file != null) ? file.getName() : "";
    throw new EcoSafariException("Opción open en construcción. Archivo " + fileName);
}
```

```
public void open(File file) throws EcoSafariException{
    throw new EcoSafariException("Opcion open esta en construccion");
}

public void saveAs(File file) throws EcoSafariException{
    throw new EcoSafariException("Opcion saveAs esta en construccion");
}

public void import_(File file) throws EcoSafariException{
    throw new EcoSafariException("Opcion import esta en construccion");
}

public void exportAs(File file) throws EcoSafariException{
    throw new EcoSafariException("Opcion exportAs esta en construccion");
}
```

Figura 3: Métodos de entrada-salida en la fachada del dominio.

2. **Vista.** El método `prepareViewMenu` construye el menú *Archivo* con las seis opciones y sus separadores.

```
private void prepareViewMenu(){
    JMenuBar menuBar = new JMenuBar();
    JMenu menuArchivo = new JMenu("Archivo");

    JMenuItem itemNuevo = new JMenuItem("Nuevo");
    JMenuItem itemAbrir = new JMenuItem("Abrir");
    JMenuItem itemSalvarComo = new JMenuItem("Salvar como");
    JMenuItem itemImportar = new JMenuItem("Importar");
    JMenuItem itemExportarComo = new JMenuItem("Exportar como");
    JMenuItem itemSalir = new JMenuItem("Salir");


    menuArchivo.add(itemNuevo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemAbrir);
    menuArchivo.add(itemSalvarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemImportar);
    menuArchivo.add(itemExportarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);

    menuBar.add(menuArchivo);
    setJMenuBar(menuBar);

}
```

Figura 4: Método prepareViewMenu.

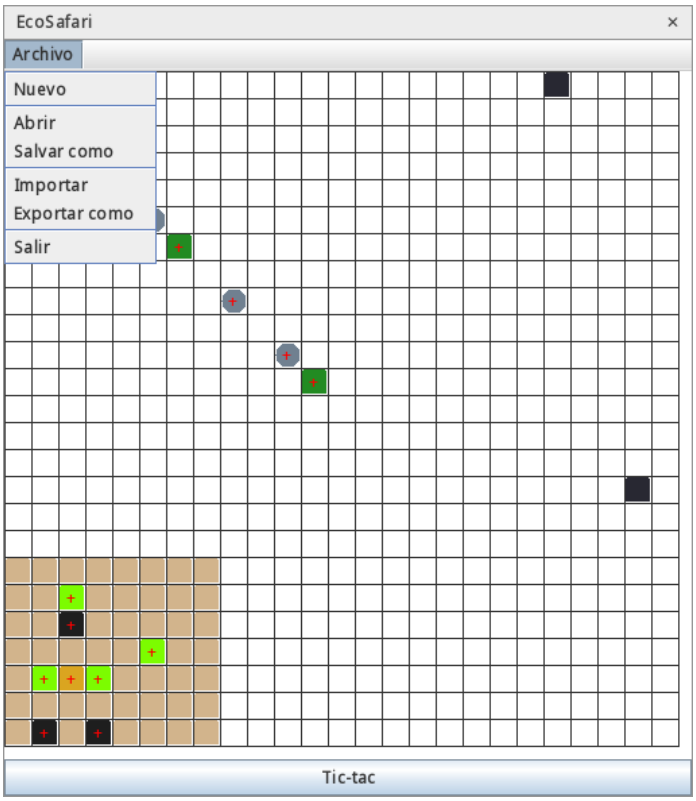


Figura 5: Menú Archivo.

3. **Controlador.** El método `prepareControllerMenu` asocia cada opción con su método: `optionNew`, `optionOpen`, `optionSaveAs`, `optionImport`, `optionExportAs` y `optionExit`.

   a) **Salir:** `optionExit` termina la aplicación con `System.exit(0)`, sin confirmación.

   b) **Nuevo:** `optionNew` reemplaza el safari actual por uno nuevo y repinta la vista.

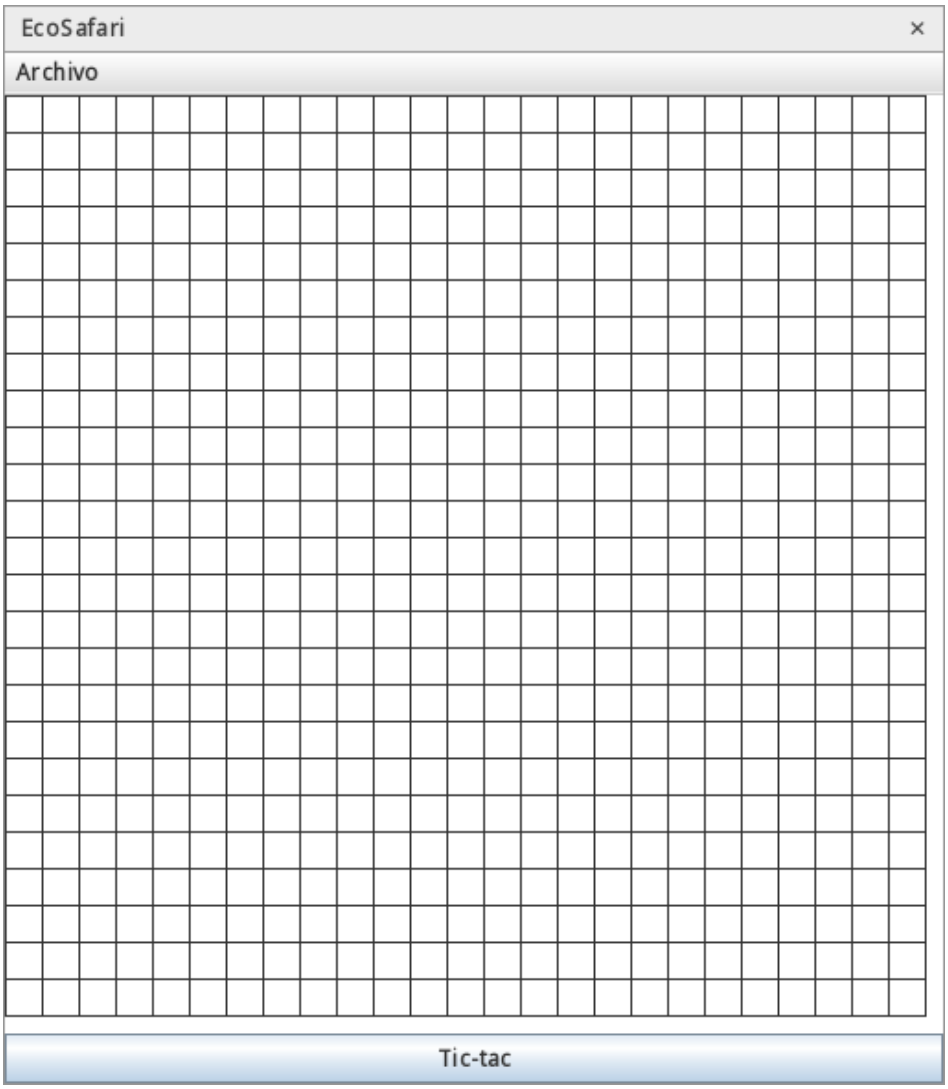


Figura 6: Resultado de la opción Nuevo.

c) **Entrada-salida:** cada opción abre un `JFileChooser`, llama al método del modelo y muestra la excepción en un `JOptionPane`.

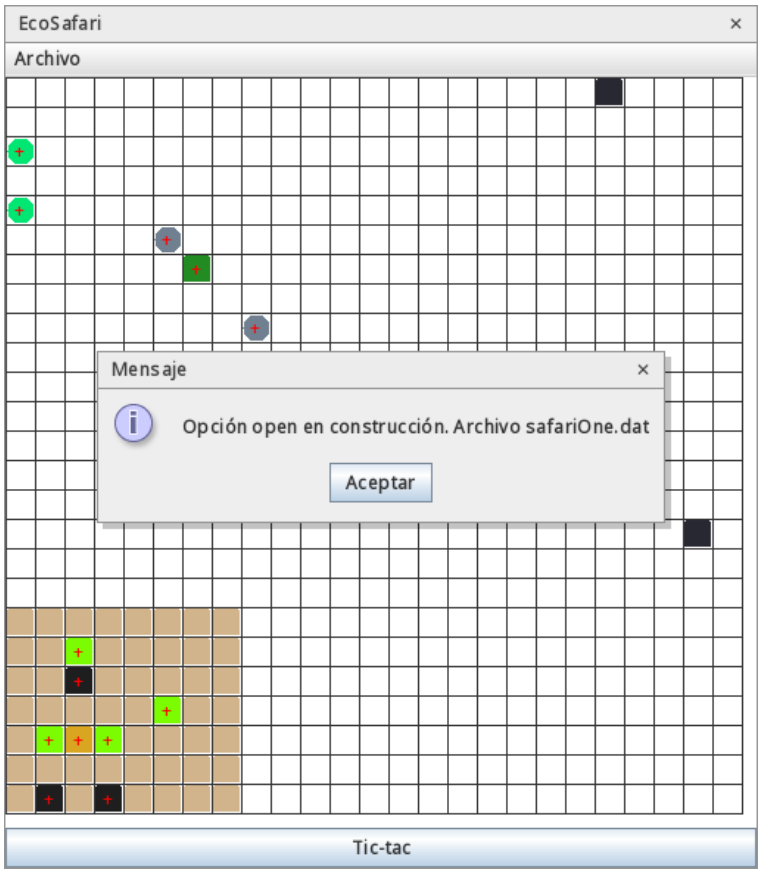


Figura 7: Abrir: mensaje de opción en construcción.

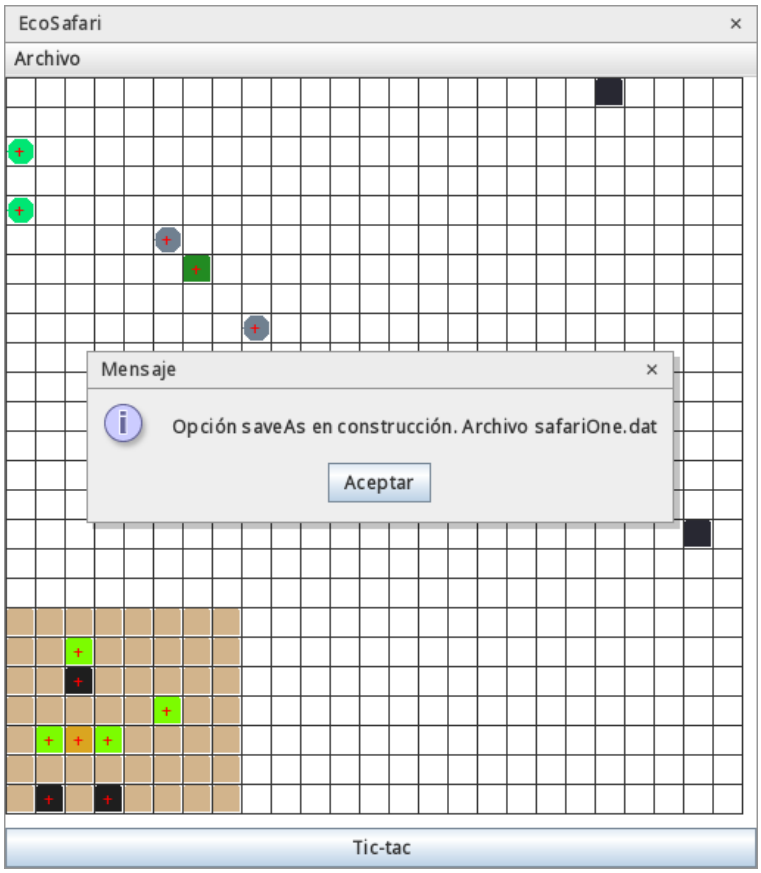


Figura 8: Salvar como: mensaje de opción en construcción.

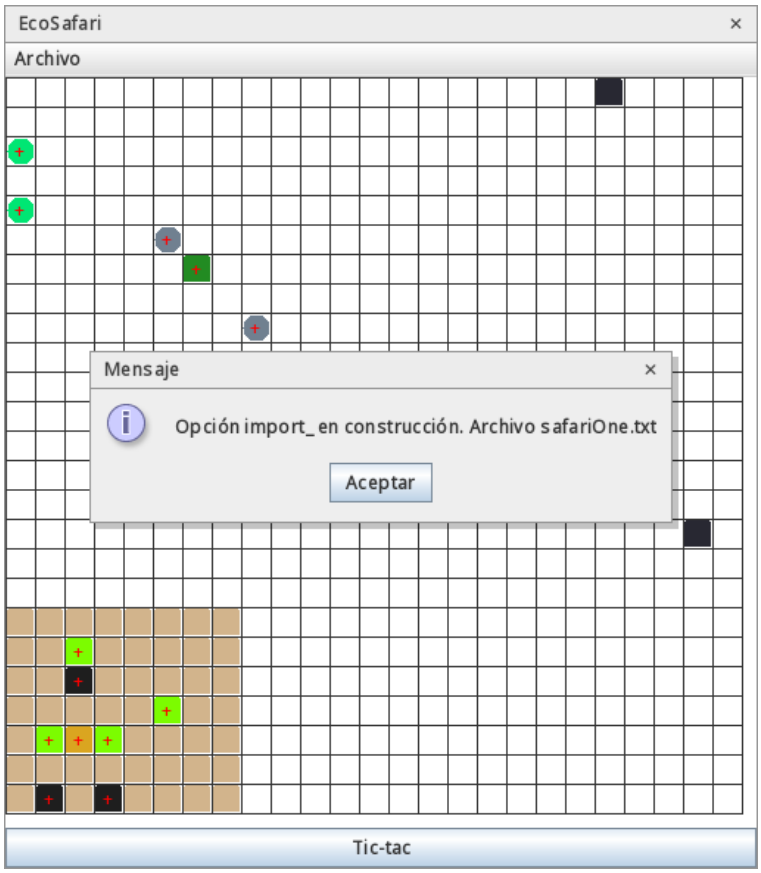


Figura 9: Importar: mensaje de opción en construcción.

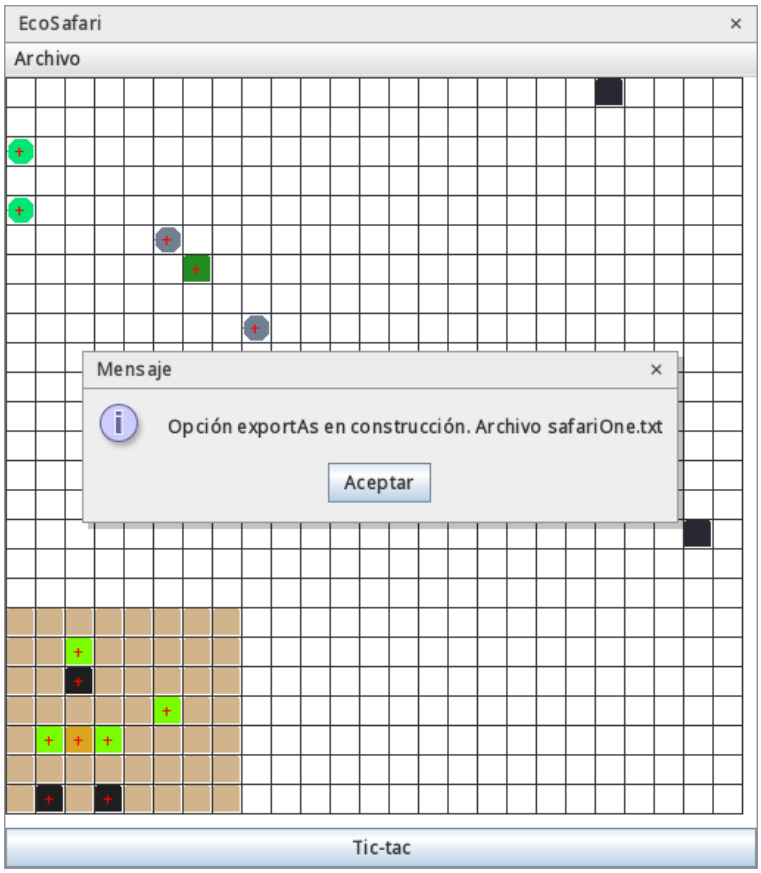


Figura 10: Exportar como: mensaje de opción en construcción.

Parte II. Implementando persistencia

A. Salvar y abrir

1. Las versiones de maqueta se conservaron como `open00` y `saveAs00`.

2. **saveAs.** Serializa el safari completo con `ObjectOutputStream`. Para ello `EcoSafari` y todas las entidades implementan `Serializable` y declaran `serialVersionUID`.

```
try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file))) {
    out.writeObject(this);
} catch (IOException e) {
    throw new EcoSafariException("el siguiente archivo no ha sido posible guardarlo "
        + file.getName());
}
```

3. *¿El archivo se creó en el disco? ¿Cuánto espacio ocupa?*

   R/ Sí. `safariOne.dat` ocupa aproximadamente 2.3 KB (2 311 bytes). El tamaño varía ligeramente entre ejecuciones porque la simulación tiene un componente aleatorio (`Ground`).

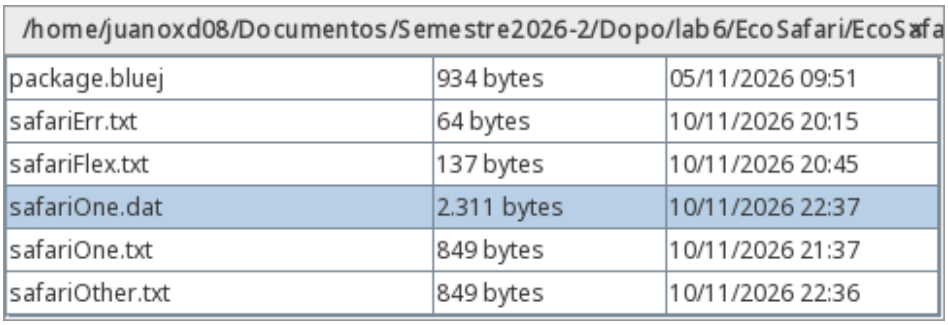


Figura 11: Archivo safariOne.dat en el disco.

4. **open.** Lee el objeto con `ObjectInputStream` y reemplaza el tablero actual por el leído.
5. *Prueba de aceptación: iniciar, crear un nuevo estado y abrir safariOne.dat.*

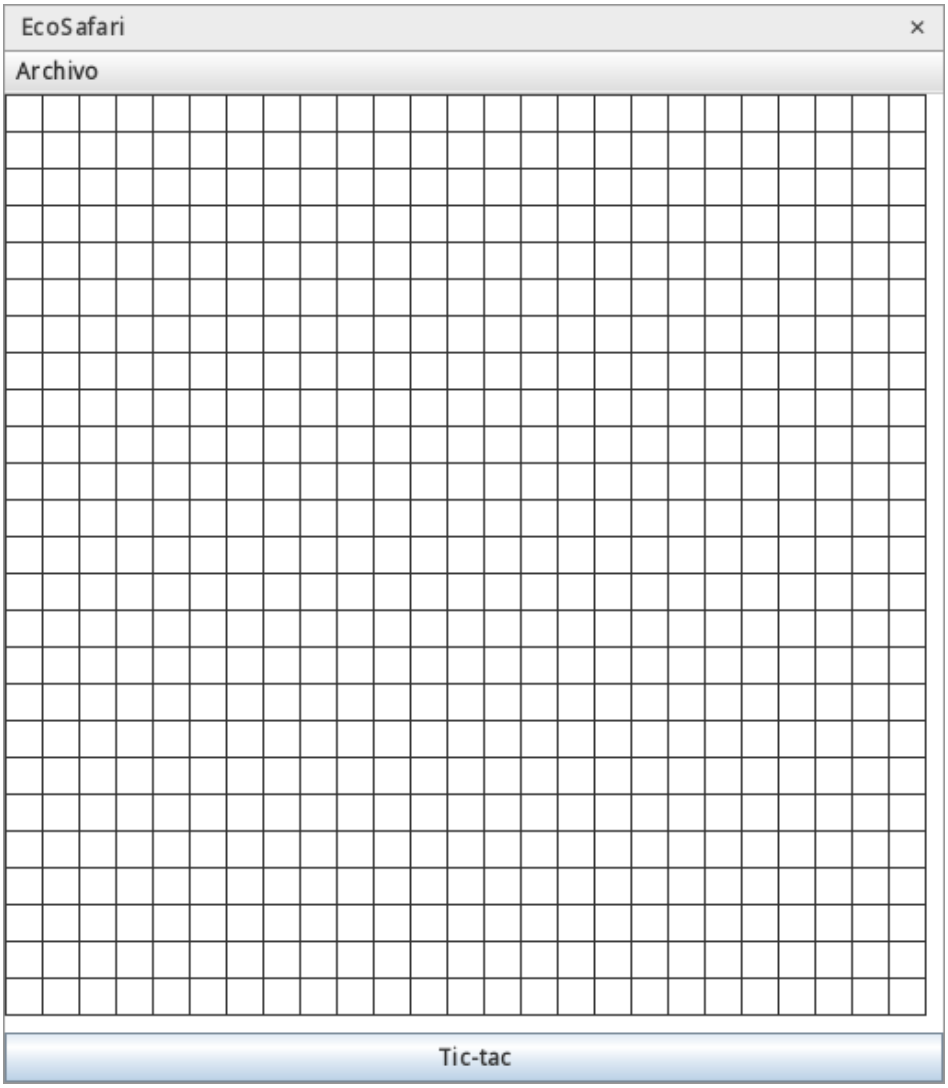


Figura 12: Nuevo estado antes de abrir.

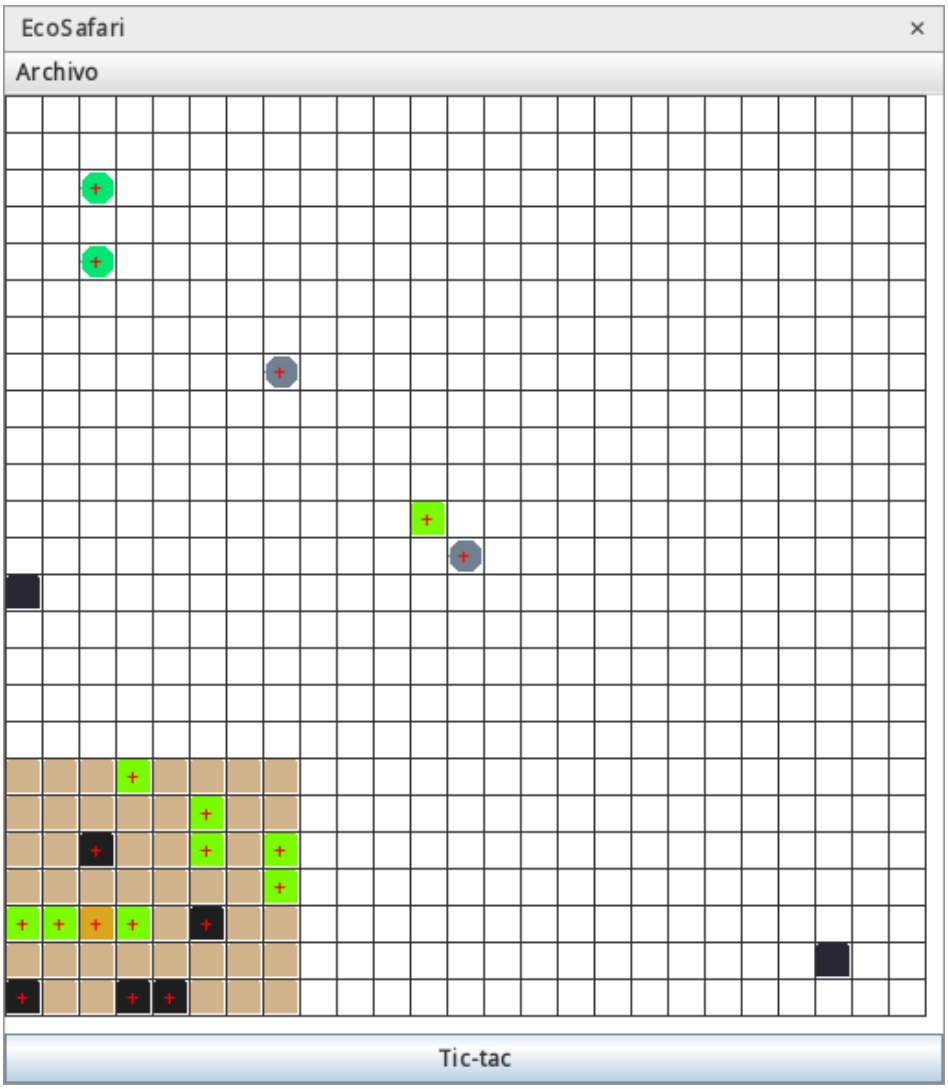


Figura 13: Estado restaurado desde safariOne.dat.

B. Exportar e importar

1. Las versiones de maqueta se conservaron como `import00` y `exportAs00`.
2. **exportAs.** Recorre el tablero y escribe una línea por entidad con el formato `Tipo fila, columna`, usando `getClass().getSimpleName()`.
3. *Exportar como safariOne.txt, editarlo y analizar. ¿Qué pasó?*

   R/ Se generó un archivo de texto legible, con una línea por cada entidad. Las celdas donde una entidad estaba sobre el suelo (por ejemplo, `Grass` sobre `Ground`) solo aparecen con la entidad visible. El archivo guarda tipo y posición, pero no el estado interno (energía).

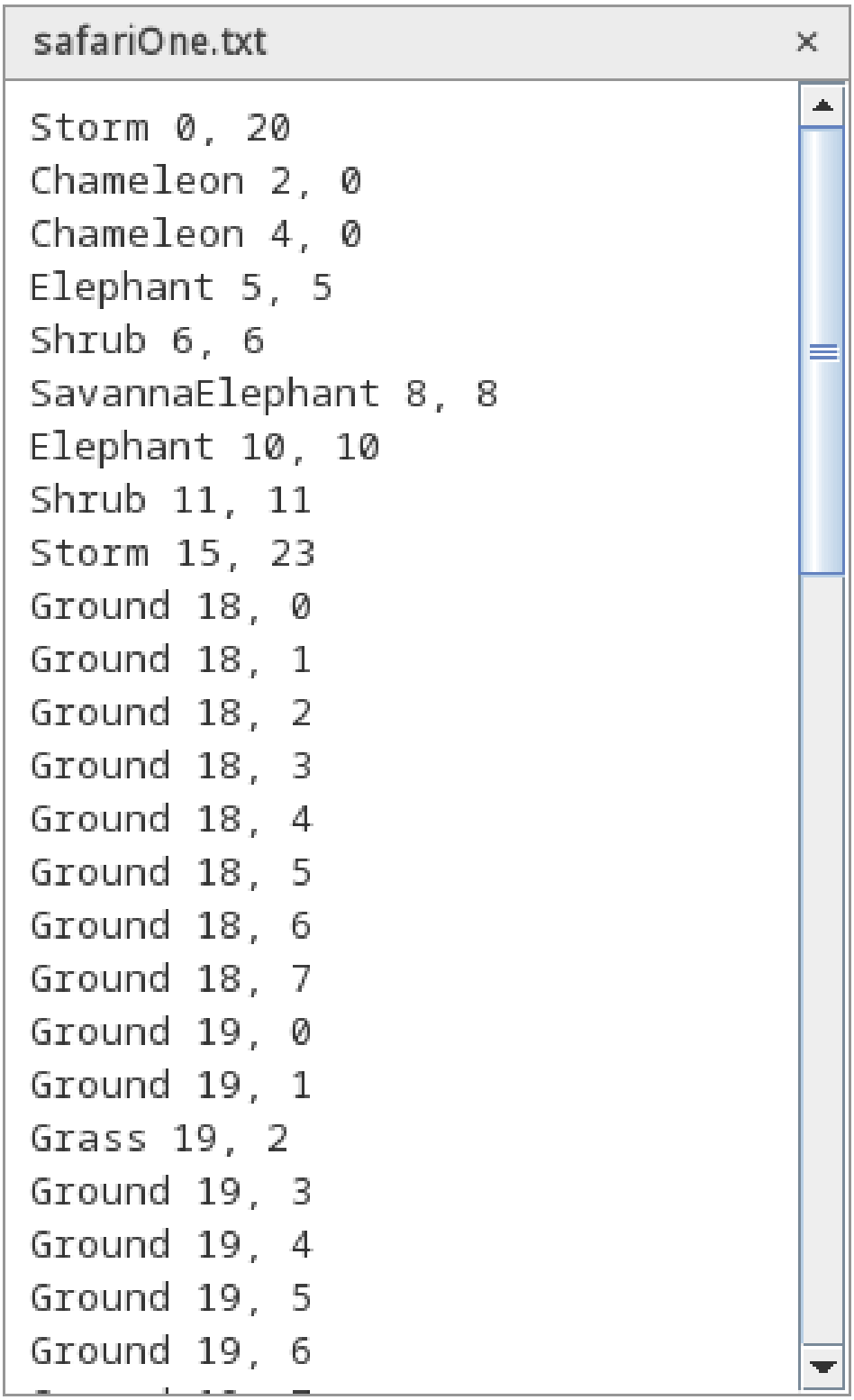


Figura 14: Contenido de safariOne.txt exportado.

4. **import_.** Lee cada línea, la separa con `trim` y `split` y crea la entidad correspondiente en la posición indicada.

5. *Exportar a safariOther.txt, salir, entrar, Nuevo e importar. ¿Qué resultado obtuvieron?*

   R/ Se recuperó el mismo tablero: mismos tipos y posiciones. Las entidades inician con su estado por defecto (energía 100), ya que el archivo de texto no lo guarda.

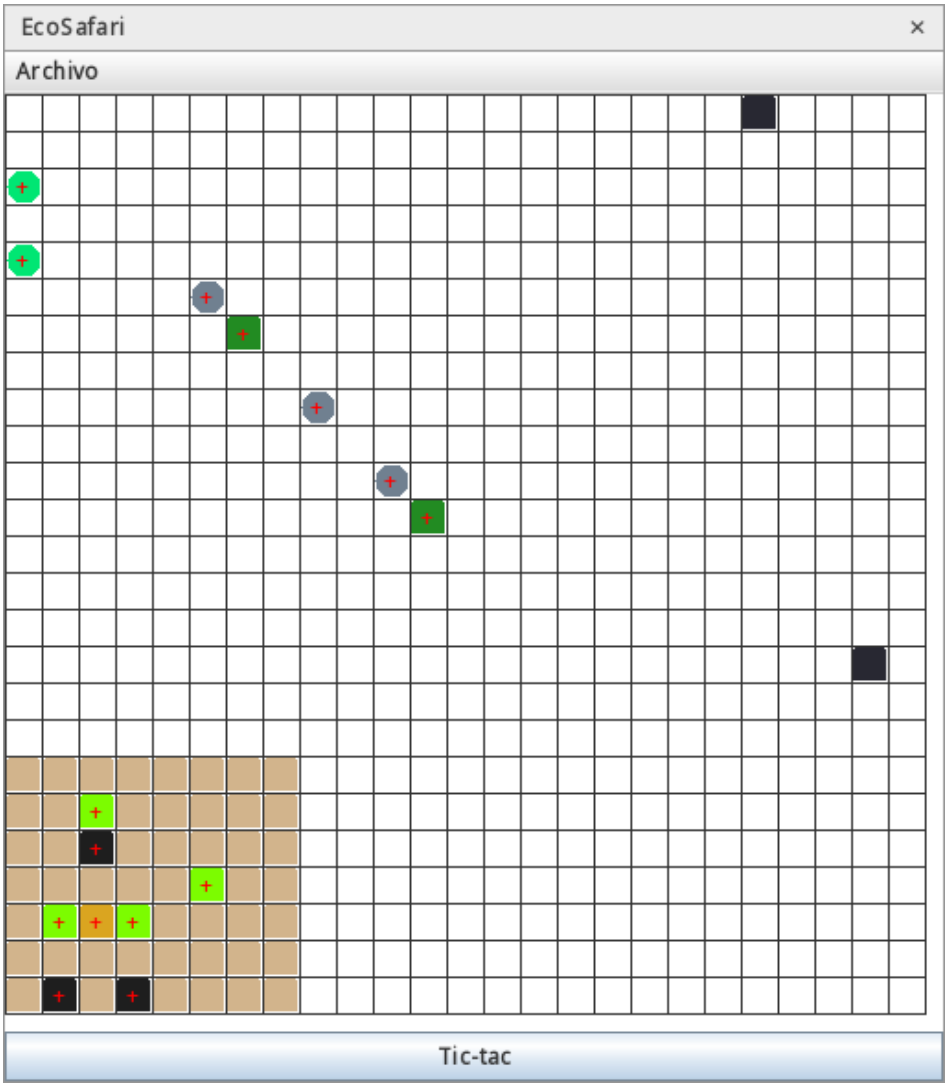


Figura 15: Safari después de importar safariOther.txt.

6. *Escribir un safariOne.txt correcto e importarlo. ¿Qué resultado obtuvieron?*

   R/ El tablero se construyó exactamente con las entidades del archivo.

```
Elephant 5, 5
Elephant 10, 10
Shrub 6, 6
Storm 0, 20
SavannaElephant 8, 8
Chameleon 2, 0
Grass 19, 2
Zebra 20, 2
Lion 22, 2
```

```
10 Ground 18, 0
11 ...
```

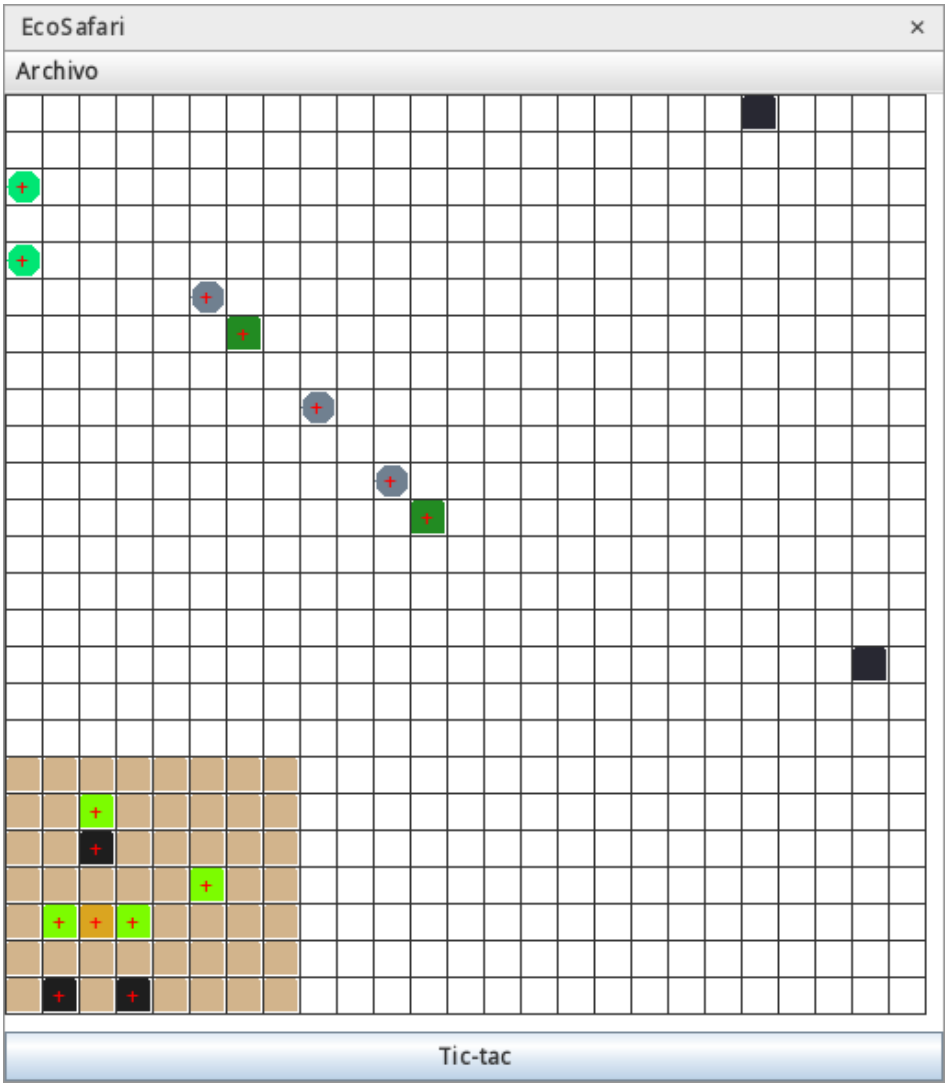


Figura 16: Safari importado desde safariOne.txt.

7. Se eliminó el método `someEntities()`. El constructor crea un safari vacío y los elementos solo se agregan mediante archivos.

C. Comportamiento

1. *Tres clics, guardar y abrir.*

   R/ Al abrir, el safari continúa exactamente donde quedó: posiciones, energía y estado de cada entidad.

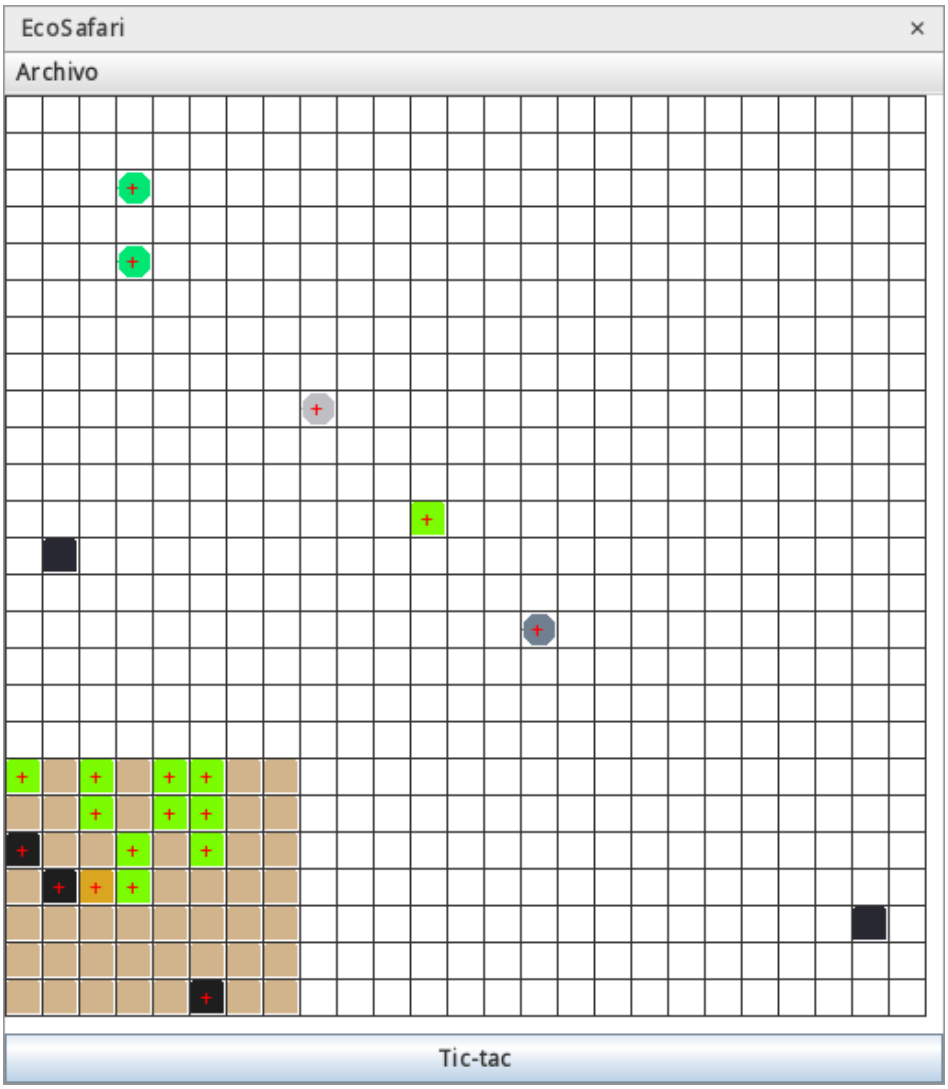


Figura 17: Safari después de guardar y abrir.

2. *Tres clics, exportar e importar.*

   R/ Se recuperan los tipos y posiciones, pero las entidades reinician su estado interno.

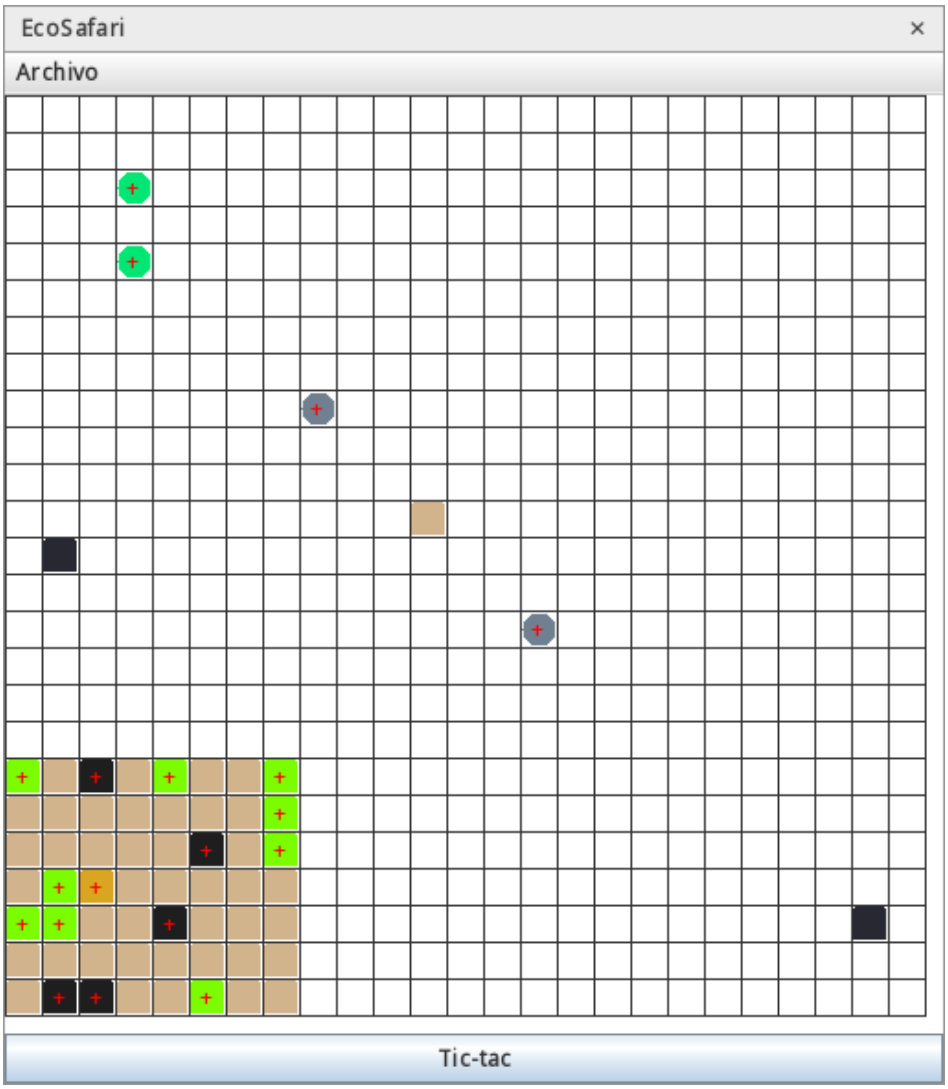


Figura 18: Safari después de exportar e importar.

3. *¿Qué diferencias ven entre 1 y 2?*

   R/ Guardar crea un archivo binario con el objeto completo, incluido el estado interno de cada entidad. Exportar crea un archivo de texto, editable a mano, que solo describe qué entidad hay y dónde. Por eso, al importar, cada entidad se construye de nuevo con sus valores iniciales, mientras que al abrir la simulación continúa sin cambios.

Parte III. Fortaleciendo la persistencia

A. Guardar y abrir

1. Las versiones con mensaje general se conservaron como `open01` y `saveAs01`.
2. Antes de leer o escribir se valida el archivo, y cada caso tiene su propio mensaje:
   - Archivo nulo: *No se ha especificado ningún archivo para abrir/guardar.*

- Archivo inexistente (`open`): *el siguiente archivo no existe.*
- Directorio: *el siguiente archivo en realidad es un directorio.*
- Extensión distinta de `.dat`: *el siguiente archivo no es un .dat.*
- Contenido inválido (`open`): *no ha sido posible abrirlo por corrupción o incompatibilidad con EcoSafari.*
- Falla de escritura (`saveAs`): *no ha sido posible guardarlo por error de escritura o permisos.*

3. *Prueba de aceptación de un nuevo mensaje.*

   R/ Se intentó abrir un archivo inexistente. Resultado: *"el siguiente archivo no existe: noexiste.dat"*.

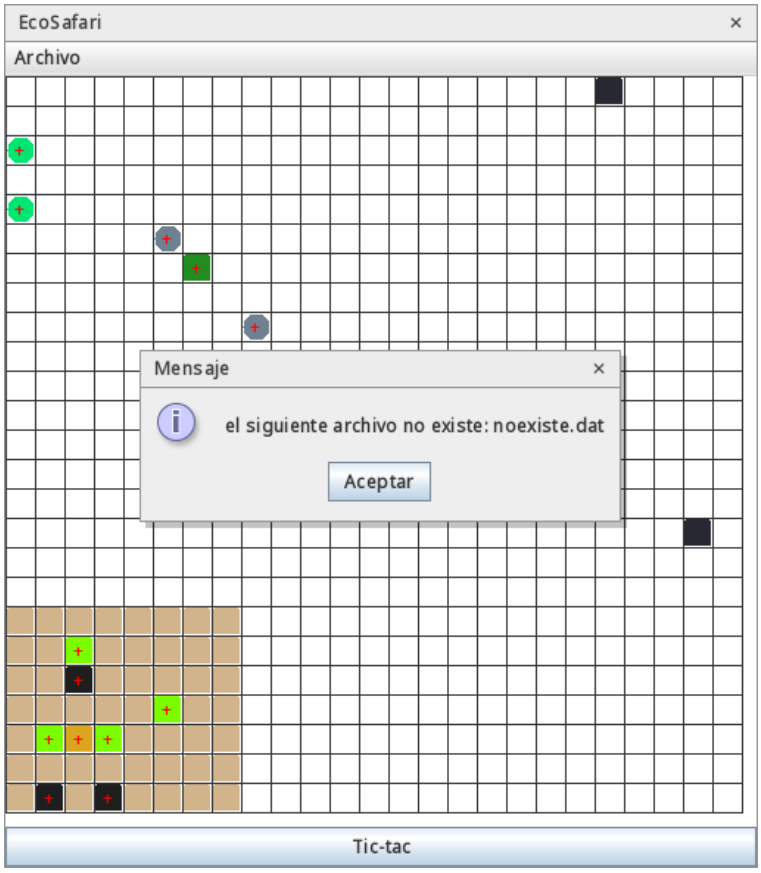


Figura 19: Mensaje de error detallado al abrir.

B. Exportar e importar

1. Las versiones con mensaje general se conservaron como `import01` y `exportAs01`.
2. Se aplicaron las mismas validaciones de archivo (nulo, inexistente, directorio y extensión `.txt`). Además, `import_` distingue líneas con datos incompletos, coordenadas no numéricas y tipos de entidad no reconocidos.
3. *Prueba de aceptación de un nuevo mensaje.*

   R/ Se intentó exportar a un archivo con extensión `.dat`. Resultado: *"el siguiente archivo no es un .txt: a.dat"*.

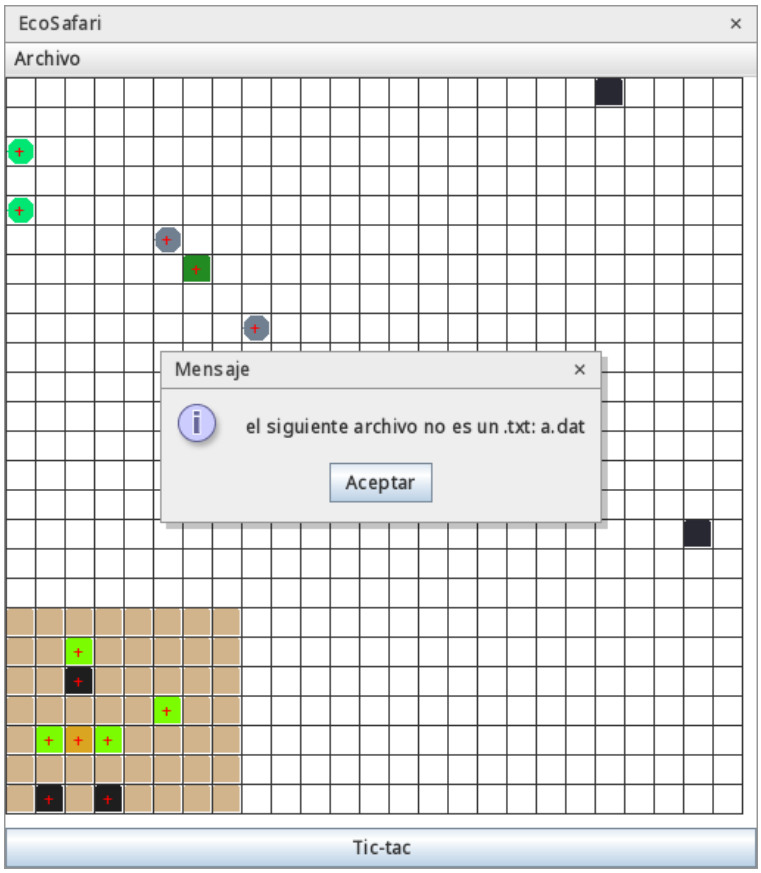


Figura 20: Mensaje de error detallado al exportar.

Pruebas de unidad

Las pruebas de persistencia están en `EcoSafariPersistenceTest` (24 pruebas) y cubren guardar y abrir, exportar e importar, cada error del minicompilador y la diferencia de estado entre ambos mecanismos: `open` conserva la energía y `import_` la reinicia. El proyecto completo suma 53 pruebas, todas exitosas.

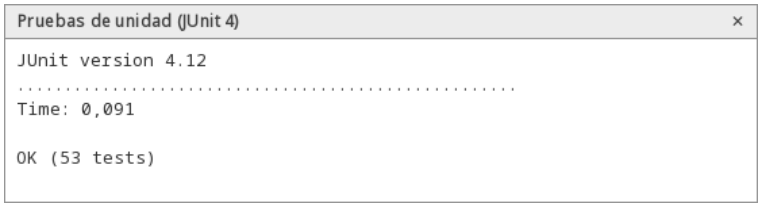


Figura 21: Ejecución de las pruebas de unidad.

Parte IV. Bono: perfeccionando importar

A. Minicompilador

1. La versión anterior se conservó como `import02`.
2. Cada error indica la línea, la palabra que lo produce y la causa, con el formato:

`Línea n: error en 'palabra'. Causa: descripción.`

Además, si alguna línea falla se conserva el tablero anterior, de modo que un archivo con errores no deja el safari a medias.

3. *safariErr.txt con mínimo tres errores diferentes.*

```
Dragon 5, 5
Elephant 10, abc
Lion 50, 50
Bush 15, 15
Storm 5, 5
```

R/ El archivo se fue corrigiendo línea a línea con ayuda del importador, que reportó en orden: la clase `Dragon` no existe; la columna `abc` no es un número entero; las coordenadas `50, 50` están fuera del tablero (0 a 24); y la clase `Bush` no existe, pues en la aplicación se llama `Shrub`.

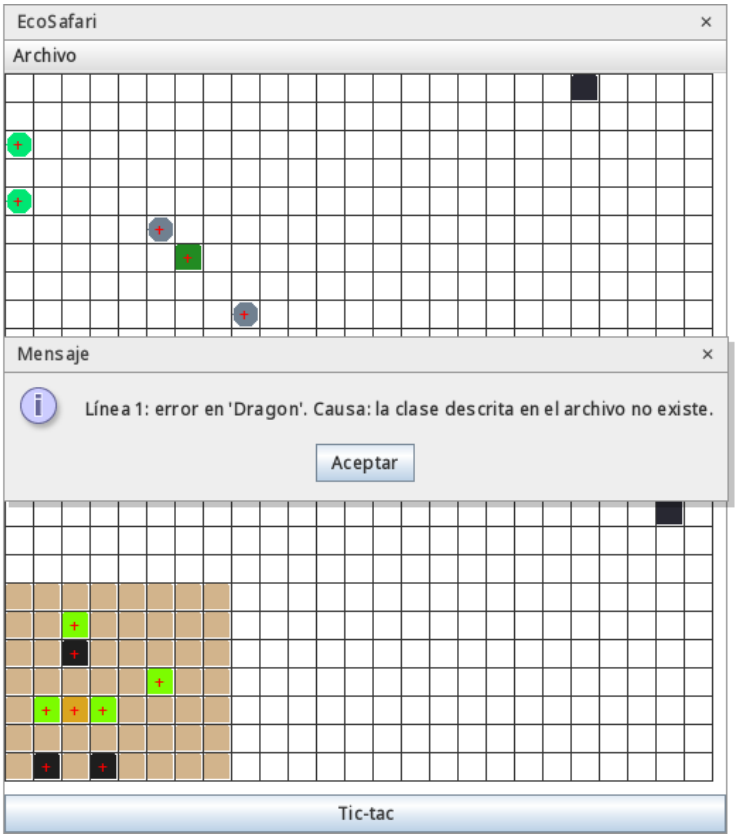


Figura 22: Error 1: clase inexistente.

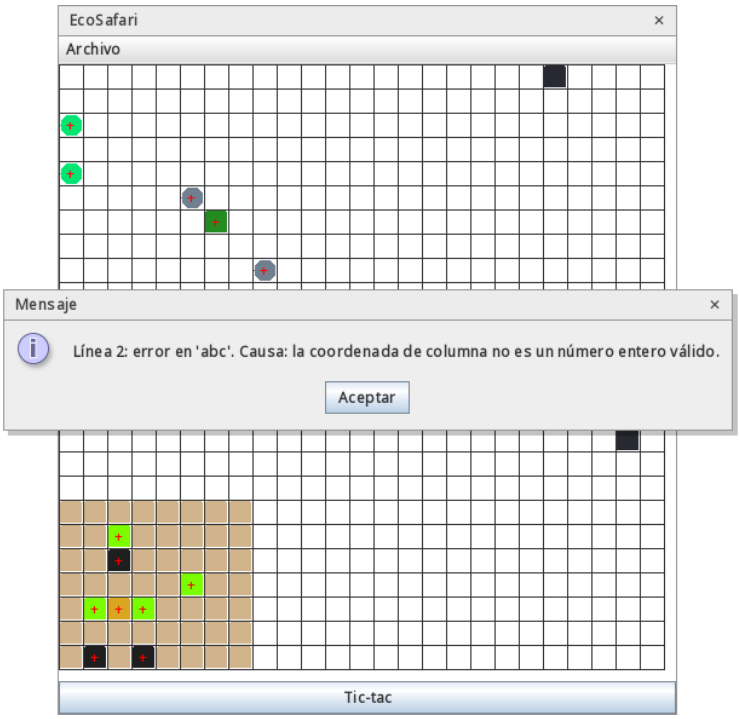


Figura 23: Error 2: coordenada no numérica.

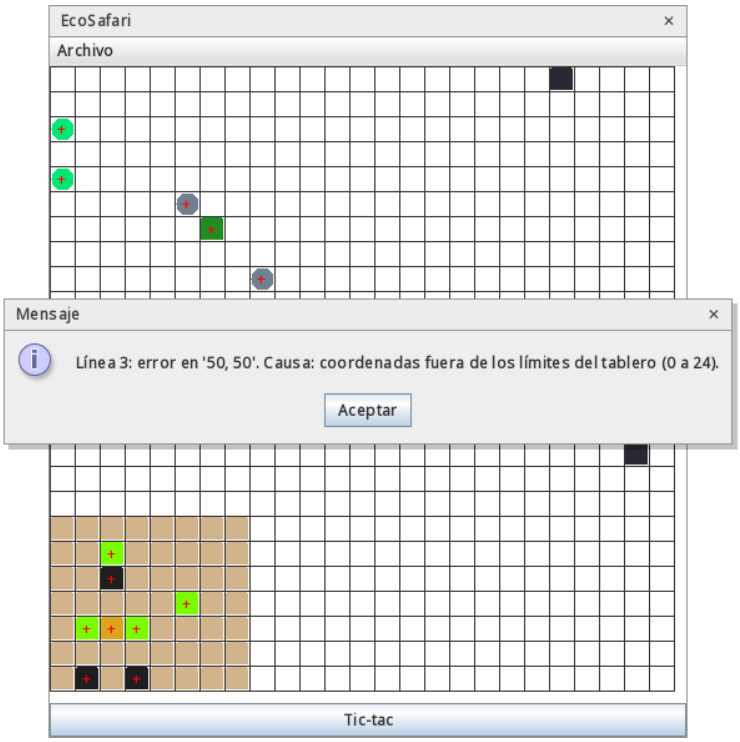


Figura 24: Error 3: coordenadas fuera de límites.

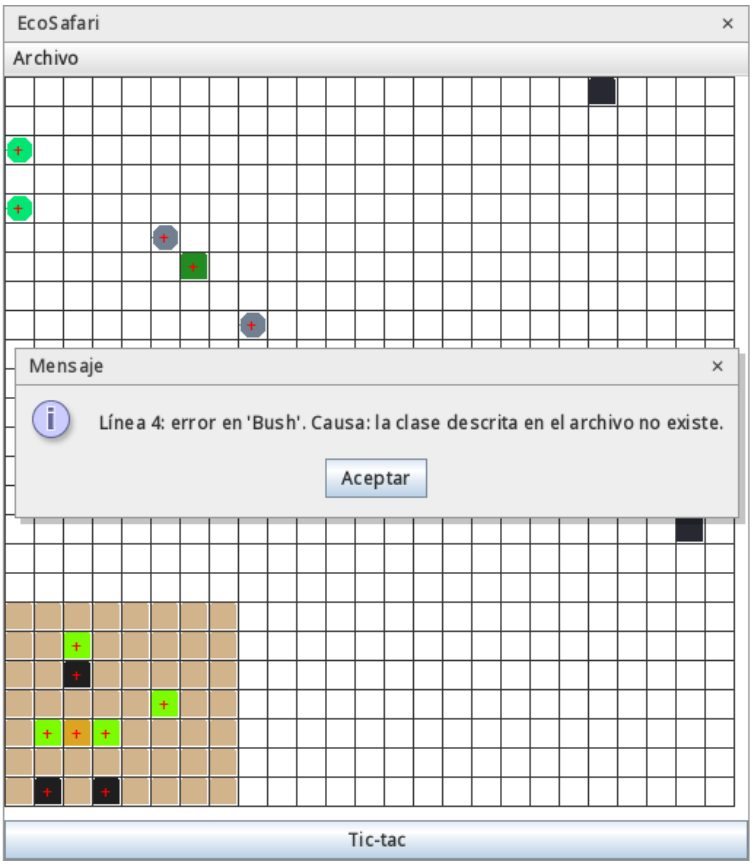


Figura 25: Error 4: nombre de clase incorrecto.

B. Minicompilador flexible

1. La versión anterior se conservó como `import03`.

2. El `switch` de tipos se reemplazó por reflexión: la clase se busca por su nombre y se invoca su constructor `(EcoSafari, int, int)`. Así cualquier entidad futura se importa sin modificar el método.

```
try {
    Class.forName("domain." + tipo)
         .getConstructor(EcoSafari.class, int.class, int.class)
         .newInstance(this, i, j);
} catch (ClassNotFoundException ce) {
    throw new EcoSafariException("Línea " + numeroLinea + ": error en '" + tipo
        + "'. Causa: la clase descrita en el archivo no existe.");
} catch (NoSuchMethodException | InstantiationException
         | IllegalAccessException | InvocationTargetException e) {
    throw new EcoSafariException("Línea " + numeroLinea + ": error en '" + tipo
        + "'. Causa: la clase descrita tiene inconvenientes en su constructor.");
}
```

3. *¿exportAs necesita cambiar para ser flexible?*

   R/ No. `exportAs` obtiene el tipo con `getClass().getSimpleName()`, por lo que ya funciona con cualquier clase nueva sin modificaciones.

4. No aplica, ya que `exportAs` no requirió cambios.

5. *safariFlex.txt para probar la flexibilidad.*

```
1  Elephant 5, 5
2  Lion 8, 8
3  Zebra 12, 12
4  Chameleon 3, 3
5  Shrub 15, 15
6  Storm 2, 20
7  Grass 19, 2
8  Ground 18, 0
9  SavannaElephant 10, 10
10 Dragon 1, 1
```

R/ Las nueve primeras clases se reconocen sin estar escritas en el código del importador. La línea 10 produce el error significativo: *"Línea 10: error en 'Dragon'. Causa: la clase descrita en el archivo no existe."*

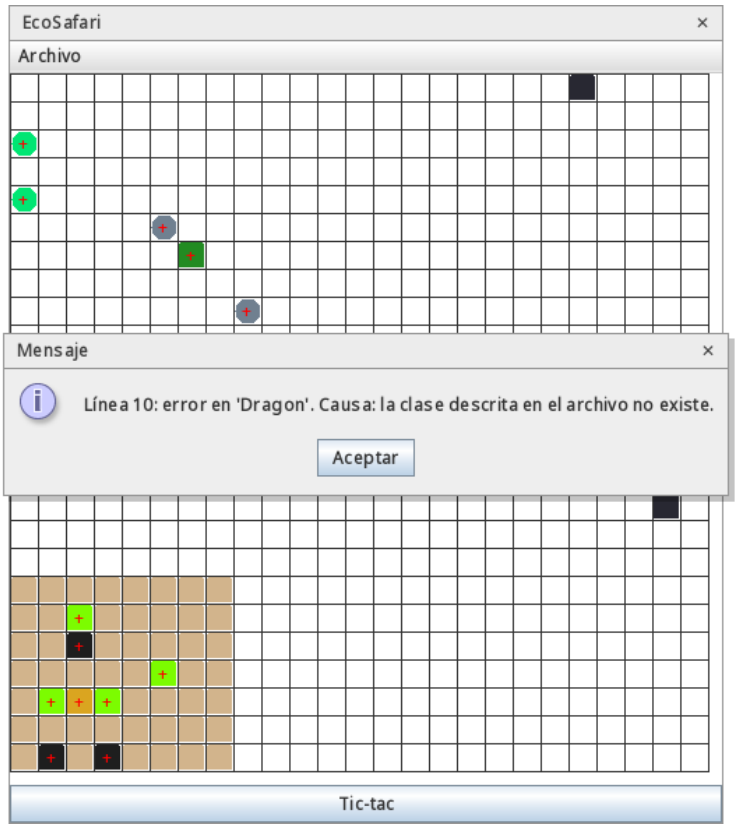


Figura 26: Error significativo al importar safariFlex.txt.

Retrospectiva

1. *¿Cuál fue el tiempo total invertido por cada uno? (Horas)*

   R/ Juan David Espitia: *5* horas. Cristian Camilo Salamanca: *3* horas.

2. *¿Cuál es el estado actual del laboratorio? ¿Por qué?*

   R/ Completo, incluido el bono. Todas las opciones del menú funcionan, los errores se reportan en detalle y las 53 pruebas de unidad pasan.

3. *¿Cuál práctica XP fue la más útil? ¿Por qué?*

   R/ La propiedad colectiva del código: ambos integrantes conocían y podían modificar cualquier clase, lo que facilitó corregir errores en el dominio y en la interfaz sin depender del otro.

4. *¿Cuál consideran fue el mayor logro? ¿Por qué?*

   R/ El minicompilador flexible: con reflexión, el importador reconoce cualquier entidad futura y reporta los errores con línea, palabra y causa.

5. *¿Cuál fue el mayor problema técnico? ¿Qué hicieron para resolverlo?*

   R/ Un archivo con errores dejaba el safari a medio cargar, porque las entidades se registran en el tablero desde su constructor. Se resolvió guardando una referencia al tablero anterior y restaurándola si la importación falla.

6. *¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar?*

   R/ Se dividió bien el trabajo entre modelo, vista y pruebas. Como mejora, se comprometen a escribir las pruebas de unidad antes de implementar cada método.

7. *¿Qué referencias usaron? ¿Cuál fue la más útil?*

   R/ La más útil fue la documentación oficial de Java sobre serialización y reflexión.

Referencias

[1] Oracle. (2024). *Java Object Serialization Specification*. Oracle Corporation. https://docs.oracle.com/en/java/javase/21/docs/specs/serialization/index.html

[2] Oracle. (2024). *Class Class<T>* [Documentación de la API de Java SE 21]. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Class.html

[3] Oracle. (2024). *The Java Tutorials: Basic I/O*. https://docs.oracle.com/javase/tutorial/essential/io/

[4] Oracle. (2024). *How to Use File Choosers*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/components/filechooser.html

[5] Beck, K., & Andres, C. (2004). *Extreme Programming Explained: Embrace Change* (2.ª ed.). Addison-Wesley.